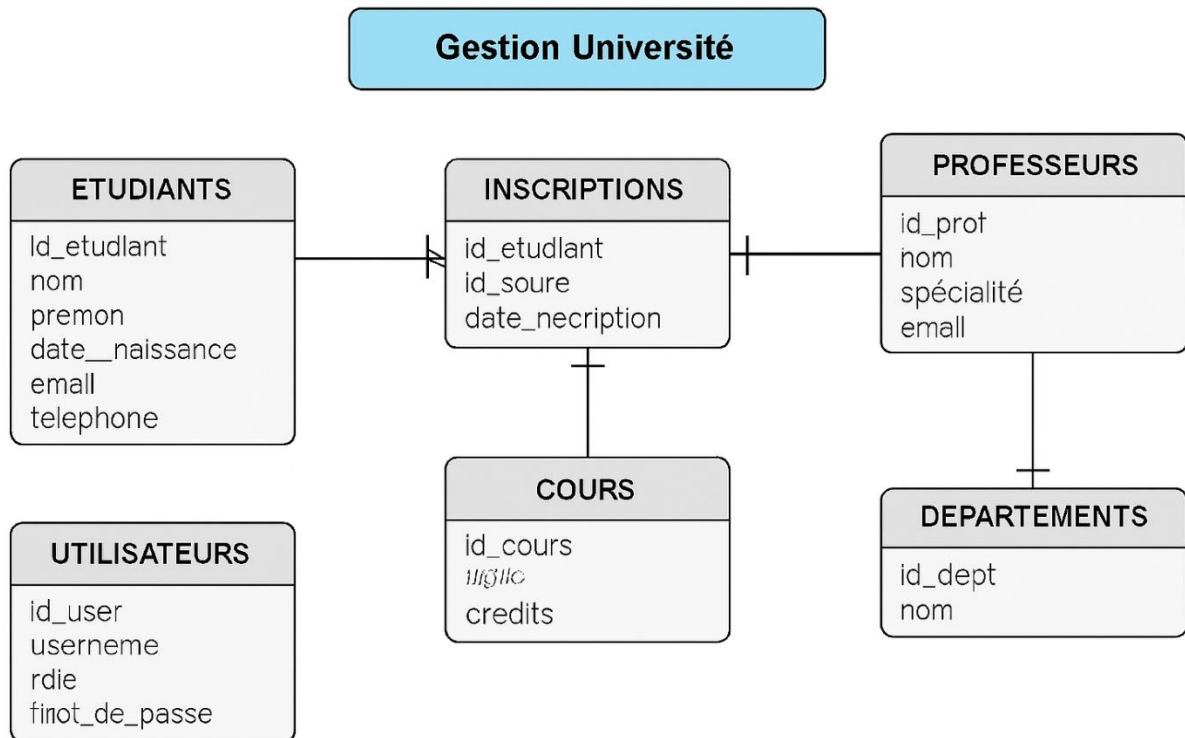


ITAC

Departement des sciences informatiques

Cours: Prf. Loveson VILSENAT

Administration de base de données Oracle



1. Création des tables avec relations

Table DEPARTEMENTS

```
CREATE TABLE DEPARTEMENTS (  
    id_dept NUMBER PRIMARY KEY,  
    nom VARCHAR2(50) UNIQUE  
);
```

Table PROFESSEURS

```
CREATE TABLE PROFESSEURS (  
    id_prof NUMBER PRIMARY KEY,  
    nom VARCHAR2(50),  
    specialite VARCHAR2(50),  
    email VARCHAR2(100) UNIQUE,  
    id_dept NUMBER,  
    FOREIGN KEY (id_dept) REFERENCES DEPARTEMENTS(id_dept)  
);
```

Table ETUDIANTS

```
CREATE TABLE ETUDIANTS (  
    id_etudiant NUMBER PRIMARY KEY,  
    nom VARCHAR2(50) NOT NULL,  
    prenom VARCHAR2(50),  
    date_naissance DATE,  
    email VARCHAR2(100) UNIQUE,  
    telephone VARCHAR2(20)  
);
```

Table COURS

```
CREATE TABLE COURS (  
    id_cours NUMBER PRIMARY KEY,  
    titre VARCHAR2(100),  
    credits NUMBER CHECK (credits BETWEEN 1 AND 10),  
    id_prof NUMBER,  
    FOREIGN KEY (id_prof) REFERENCES PROFESSEURS(id_prof)  
);
```

Table INSCRIPTIONS

```
CREATE TABLE INSCRIPTIONS (  
    id_etudiant NUMBER,  
    id_cours NUMBER,  
    date_inscription DATE NOT NULL,  
    note NUMBER,  
    PRIMARY KEY (id_etudiant, id_cours),  
    FOREIGN KEY (id_etudiant) REFERENCES ETUDIANTS(id_etudiant),  
    FOREIGN KEY (id_cours) REFERENCES COURS(id_cours)  
);
```

Table UTILISATEURS

```
CREATE TABLE UTILISATEURS (  
    id_user NUMBER PRIMARY KEY,  
    username VARCHAR2(50) UNIQUE,  
    role VARCHAR2(20) CHECK (role IN ('ADMIN', 'ETUDIANT', 'PROF')),  
    mot_de_passe VARCHAR2(100)  
);
```

2. Insertions pour chacune des tables

DEPARTEMENTS

```
INSERT INTO DEPARTEMENTS VALUES (1, 'Mathématiques');
INSERT INTO DEPARTEMENTS VALUES (2, 'Informatique');
INSERT INTO DEPARTEMENTS VALUES (3, 'Physique');
INSERT INTO DEPARTEMENTS VALUES (4, 'Chimie');
INSERT INTO DEPARTEMENTS VALUES (5, 'Biologie');
INSERT INTO DEPARTEMENTS VALUES (6, 'Statistiques');
INSERT INTO DEPARTEMENTS VALUES (7, 'Économie');
```

PROFESSEURS

```
INSERT INTO PROFESSEURS VALUES
(1, 'Dupont', 'Mathématiques', 'dupont@univ.com', 1);
INSERT INTO PROFESSEURS VALUES
(2, 'Martin', 'Informatique', 'martin@univ.com', 2);
INSERT INTO PROFESSEURS VALUES (3, 'Durand', 'Physique', 'durand@univ.com', 3);
INSERT INTO PROFESSEURS VALUES (4, 'Bernard', 'Chimie', 'bernard@univ.com', 4);
INSERT INTO PROFESSEURS VALUES (5, 'Petit', 'Biologie', 'petit@univ.com', 5);
INSERT INTO PROFESSEURS VALUES
(6, 'Robert', 'Statistiques', 'robert@univ.com', 6);
INSERT INTO PROFESSEURS VALUES (7, 'Laurent', 'Économie', 'laurent@univ.com', 7);
```

ETUDIANTS

```
INSERT INTO ETUDIANTS VALUES (1, 'Jean', 'Pierre', DATE '2000-05-
12', 'jean.pierre@email.com', '509111111');
INSERT INTO ETUDIANTS VALUES (2, 'Marie', 'Joseph', DATE '1999-07-
23', 'marie.joseph@email.com', '509222222');
INSERT INTO ETUDIANTS VALUES (3, 'Paul', 'Henri', DATE '2001-01-
15', 'paul.henri@email.com', '509333333');
INSERT INTO ETUDIANTS VALUES (4, 'Luc', 'Andre', DATE '2000-09-
09', 'luc.andre@email.com', '509444444');
INSERT INTO ETUDIANTS VALUES (5, 'Sophie', 'Durand', DATE '1998-11-
30', 'sophie.durand@email.com', '509555555');
INSERT INTO ETUDIANTS VALUES (6, 'Marc', 'Louis', DATE '2002-03-
05', 'marc.louis@email.com', '509666666');
INSERT INTO ETUDIANTS VALUES (7, 'Claire', 'Jean', DATE '1999-12-
25', 'claire.jean@email.com', '509777777');
```

COURS

```
INSERT INTO COURS VALUES (1, 'Algèbre', 5, 1);
INSERT INTO COURS VALUES (2, 'Programmation Java', 6, 2);
INSERT INTO COURS VALUES (3, 'Physique Générale', 4, 3);
INSERT INTO COURS VALUES (4, 'Chimie Organique', 5, 4);
INSERT INTO COURS VALUES (5, 'Biologie Cellulaire', 3, 5);
INSERT INTO COURS VALUES (6, 'Statistiques', 4, 6);
INSERT INTO COURS VALUES (7, 'Microéconomie', 4, 7);
```

INSCRIPTIONS

```
INSERT INTO INSCRIPTIONS VALUES (1, 1, DATE '2023-09-01', 85);
INSERT INTO INSCRIPTIONS VALUES (2, 2, DATE '2023-09-01', 90);
INSERT INTO INSCRIPTIONS VALUES (3, 3, DATE '2023-09-01', 70);
INSERT INTO INSCRIPTIONS VALUES (4, 4, DATE '2023-09-01', 65);
INSERT INTO INSCRIPTIONS VALUES (5, 5, DATE '2023-09-01', 88);
INSERT INTO INSCRIPTIONS VALUES (6, 6, DATE '2023-09-01', 92);
INSERT INTO INSCRIPTIONS VALUES (7, 7, DATE '2023-09-01', 75);
```

UTILISATEURS

```
INSERT INTO UTILISATEURS VALUES (1, 'admin1', 'ADMIN', 'pwdadmin');
INSERT INTO UTILISATEURS VALUES (2, 'etudiant1', 'ETUDIANT', 'pwd1');
```

```
INSERT INTO UTILISATEURS VALUES (3, 'etudiant2', 'ETUDIANT', 'pwd2');
INSERT INTO UTILISATEURS VALUES (4, 'etudiant3', 'ETUDIANT', 'pwd3');
INSERT INTO UTILISATEURS VALUES (5, 'prof1', 'PROF', 'pwdprof1');
INSERT INTO UTILISATEURS VALUES (6, 'prof2', 'PROF', 'pwdprof2');
INSERT INTO UTILISATEURS VALUES (7, 'admin2', 'ADMIN', 'pwdadmin2');
```

Objectif de ce projet

Former les étudiants à la maîtrise des concepts fondamentaux d'Oracle Database à travers un projet réaliste et progressif : **Gestion d'une Université** (étudiants, professeurs, cours, inscriptions, utilisateurs, départements).

Tables du projet

- **ETUDIANTS** : informations personnelles des étudiants
- **PROFESSEURS** : enseignants et spécialités
- **COURS** : cours proposés avec crédits et professeur responsable
- **INSCRIPTIONS** : relation entre étudiants et cours avec notes
- **UTILISATEURS** : gestion des comptes et rôles (ADMIN, PROF, ETUDIANT)
- **DEPARTEMENTS** : rattachement des professeurs à des départements

Partie 1 : Création de tables et contraintes (10)

1. Créez la table **ETUDIANTS** avec une contrainte UNIQUE sur l'email.
2. Ajoutez une contrainte CHECK sur les crédits des cours.
3. Créez la table **INSCRIPTIONS** avec une clé composite.
4. Ajoutez une contrainte NOT NULL sur la date d'inscription.
5. Créez la table **UTILISATEURS** avec rôle limité à ('ADMIN', 'ETUDIANT', 'PROF').
6. Ajoutez un champ téléphone à la table **ETUDIANTS**.
7. Supprimez une contrainte et recréez-la.
8. Créez un index sur l'email des étudiants.
9. Créez la table **DEPARTEMENTS** et reliez-la aux professeurs.
10. Ajoutez une contrainte pour que chaque professeur appartienne à un département.

Partie 2 : Requêtes SQL de base (15)

11. Affichez tous les étudiants.
12. Listez les cours avec leurs professeurs.

13. Trouvez les étudiants inscrits à un cours donné.
14. Comptez le nombre d'étudiants par cours.
15. Affichez les étudiants sans inscription.
16. Recherchez les cours avec plus de 3 crédits.
17. Triez les étudiants par date de naissance.
18. Affichez les notes moyennes par cours.
19. Trouvez les étudiants ayant une note supérieure à 80.
20. Faites une jointure entre **ETUDIANTS** et **INSCRIPTIONS**.
21. Utilisez une sous-requête pour trouver les meilleurs étudiants.
22. Affichez les cours sans professeur.
23. Utilisez **GROUP BY** pour regrouper par professeur.
24. Utilisez **HAVING** pour filtrer les cours avec plus de 10 étudiants.
25. Créez une vue affichant les étudiants et leurs cours.

Partie 3 : Gestion des utilisateurs et sessions (10)

26. Créez un utilisateur **etudiant1** avec mot de passe.
27. Attribuez le rôle **ETUDIANT** à cet utilisateur.
28. Créez un utilisateur **prof1** avec rôle PROF.
29. Créez un utilisateur **admin1** avec rôle ADMIN.
30. Vérifiez les privilèges d'un utilisateur.
31. Accordez le droit de **SELECT** sur la table **COURS** à un étudiant.
32. Révoquez un privilège accordé à un utilisateur.
33. Créez une session et connectez-vous avec un utilisateur.
34. Vérifiez les sessions actives dans la base.
35. Fermez une session utilisateur.

Partie 4 : Procédures et PL/SQL (15)

36. Créez une procédure pour inscrire un étudiant à un cours.
37. Créez une fonction qui calcule la moyenne d'un étudiant.
38. Créez une procédure qui supprime un étudiant et ses inscriptions.
39. Créez un trigger qui empêche l'insertion d'une note > 100.
40. Créez un trigger qui logge les inscriptions dans une table **LOG_INSCRIPTIONS**.

41. Créez une séquence pour générer les IDs des étudiants.
42. Utilisez un curseur pour parcourir les étudiants.
43. Créez une procédure qui affiche les cours d'un professeur.
44. Créez une fonction qui retourne le nombre d'étudiants par cours.
45. Créez une procédure qui met à jour la note d'un étudiant.
46. Créez un package regroupant les procédures liées aux inscriptions.
47. Créez une exception personnalisée pour gérer les doublons.
48. Créez une procédure qui affiche les étudiants sans cours.
49. Créez une fonction qui retourne le meilleur étudiant.
50. Créez une procédure qui archive les étudiants diplômés.

Partie 5 : Questions avancées et pratiques (10)

51. Créez une vue affichant les cours avec le nombre d'étudiants inscrits.
52. Créez une procédure qui calcule la moyenne générale de l'université.
53. Créez un trigger qui empêche la suppression d'un professeur s'il a des cours.
54. Créez une fonction qui retourne le nombre de cours enseignés par un professeur.
55. Créez une procédure qui affiche les étudiants ayant échoué (note < 50).
56. Créez un package regroupant les fonctions de calcul de moyennes.
57. Créez une séquence pour générer les IDs des cours.
58. Créez une procédure qui supprime toutes les inscriptions d'un cours donné.
59. Créez une fonction qui retourne le nombre d'étudiants par département.
60. Créez une procédure qui transfère un professeur d'un département à un autre.

Partie 6 :

Créer au moins deux Tablespace :

- 1) Gestion_etudiant
- 2) Gestion_compte
- 3) Acceder a la système dans le tablespace gestion_etudiant
- 4) Mettre le tablespace gestion_compte en mode offline
- 5) Mettre le tablespace, gestion_etudiant en mode offline
- 6) Effectuer une requete de selection en utilisant le tablespace gestion_etudiant
- 7) Puis quel remarque , faites-vous

- 8) Utiliser assistant oracle pour créer une nouvelle base de données , puis importer les tables de la base dans cette nouvelle base de données
- 9) Connecter la nouvelle a la base avec Dbeaver .

Consignes

Groupe de 3 à 5 personnes

Date de soumission : 29 juillet 2026.